

Course: Image Processing 31651

Assignment #31

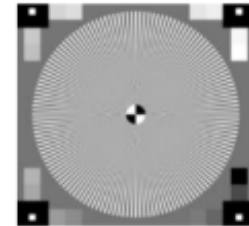
Geometrical Transforms

	ID (4 last digits)	Shorten Name	Photo of the student	
Student #1	1950	shienfeld		
Student #2	2210	pony		
Student #3	7939	akimov		

Assignment #31: Geometrical Transform

In the file ImProcInPlainC.h defined numerical values of:
NUMBER_OF_ROWS and NUMBER_OF_COLUMNS

#	Image Description
1	Calculate parameters of Affine transform that will do the following SEQUENCE of operation: Scale an image in the X direction by a factor $S_x = 1.13$ Move it 40 pixels LEFT Rotate it clockwise to 17 degrees around the point $\{x = 20, y = 10\}$ Scale an image in the y direction by a factor $S_y = 2.13$ By using Nearest Neighbor Interpolation implement this Affine transform for the image "grayImage13.bmp" Store result as A31.bmp
2	Use the same Affine Transform for the transformation of the Image "grayImage13.bmp" by using bilinear interpolation Store result as A32.bmp
3	Properly explain sequence of operation and matrix/transforms used. Compare results. Provide explanations
	For report use YOUR OWN STYLE. From YOUR Power Point customer must understand what was done, how and what are the results REMOVE ALL RED when the items are ready Important hint: Fill report pages while working. Do not complain "not enough time" in case you did not follow THIS rule.



31.1 – mathematical background about affine transformations-part 1

We are going to use several affine transformations according to the demand of the task – all the transformations (operations) will be modeled using matrix operations :

Scaling in the X Direction:

Scaling changes the size of the image in one or more directions. For scaling in the X direction by a factor S_x :

$$S_x = \begin{pmatrix} S_x & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Scaling in the Y Direction:

same concept as scaling in the X direction . For scaling in the Y direction by a factor S_y :

$$S_y = \begin{pmatrix} 1 & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Rotation Around a Specific Point (x, y):

To rotate around a point (x, y) we must move the image so that (x, y) becomes the origin, apply the rotation, and then move back.

$$ROTATION MATRIX = R = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Moving:

moves every point of an image by a certain offset. For a translation by t_x pixels right and t_y pixels up:

$$T = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

Therefore – for moving down or left we use negative sign

31.1 – mathematical background about affine transformations-part 2



The total matrix for the operation of “Rotate it clockwise to 17 degrees around the point $\{x = 20, y = 10\}$ ” will be comprised of the 3 matrix (=3 steps)



$$T_{(-x,-y)} = \begin{pmatrix} 1 & 0 & -x \\ 0 & 1 & -y \\ 0 & 0 & 1 \end{pmatrix} \text{ for moving point } (x, y) \text{ to the origin } (0,0)$$

$$R = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix} \text{ for applying the rotation (angle in minus because we rotate clockwise)}$$

$$T_{(x,y)} = \begin{pmatrix} 1 & 0 & x \\ 0 & 1 & y \\ 0 & 0 & 1 \end{pmatrix} \text{ for moving point } (x, y) \text{ from the origin } (0,0) \text{ back to } (x, y)$$



The total matrix operation for the rotation is : $R_{around (x,y)} = T_{(x,y)} * R * T_{(-x,-y)}$

31.1 – mathematical background about affine transformations-part 3



The total matrix for the operation of will be comprised of the 6 matrix (=6 steps):

$$A = S_y * T_{(x,y)} * R * T_{(-x,-y)} * T * S_x$$



The task is : “Scale an image in the X direction by a factor $S_x = 1.13$, Move it 40 pixels LEFT , Rotate it clockwise to 17 degrees around the point $\{x = 20, y = 10\}$, Scale an image in the y direction by a factor $S_y = 2.13$.



The total matrix for the operation with final numbers will be :

$$A = S_{2.13} * T_{(20,10)} * R * T_{(-20,-10)} * T * S_{1.13}$$

In addition, the angle in the cos and sin functions of the rotation matrix has to be in radians – so: $\theta = -17^\circ = -\frac{17\pi}{180} [rad]$

31.2 – implementation of the Nearest Neighbor Interpolation in code

Nearest Neighbor Interpolation is the simplest interpolation method. When transforming an image, it selects the value of the nearest pixel to the transformed coordinates.



Essentially, it rounds the transformed coordinates to the nearest integer and uses the pixel value at that location.

```
// Nearest neighbor interpolation: round to nearest integer
newCol = round(transformedCol);
newRow = round(transformedRow);

// Copy pixel value if within bounds
if (newCol >= 0 && newCol < NUMBER_OF_COLUMNS && newRow >= 0 && newRow < NUMBER_OF_ROWS)
{
    dst[row][col] = src[newRow][newCol];
}
else
{
    dst[row][col] = 0; // Assign black color to pixels out of bounds
}
```

Round the transformed coordinates to the nearest integers. The rounding lead to the selection to the nearest pixel in the input image.

Finally, copy the pixel value from the nearest neighbor in the input image to the current position in the output image.

31.3 – implementation of the bilinear interpolation in code

7

The bilinear interpolation calculates the intensity of a pixel in the transformed image by interpolating between the intensities of the four nearest pixels in the source image. This provides a smoother and more accurate result compared to the nearest neighbor interpolation.

```
95 // Calculate the coordinates of the four nearest pixels
96 int x0 = floor(transformedCol);
97 int x1 = x0 + 1;
98 int y0 = floor(transformedRow);
99 int y1 = y0 + 1;

101 // Ensure the coordinates are within the bounds of the source image
102 if (x0 >= 0 && x1 < NUMBER_OF_COLUMNS && y0 >= 0 && y1 < NUMBER_OF_ROWS)
103 {
104     // Calculate the distances
105     double dx = transformedCol - x0;
106     double dy = transformedRow - y0;
107
108     // Get the values of the four nearest pixels
109     double I00 = src[y0][x0];
110     double I10 = src[y0][x1];
111     double I01 = src[y1][x0];
112     double I11 = src[y1][x1];
113
114     // Perform bilinear interpolation
115     double I0 = I00 * (1 - dx) + I10 * dx;
116     double I1 = I01 * (1 - dx) + I11 * dx;
117     double I = I0 * (1 - dy) + I1 * dy;
118
119     // Assign the interpolated value to the destination image
120     dst[row][col] = round(I);
121 }
122 else
123 {
124     dst[row][col] = 0; // Assign black color to pixels out of bounds
125 }
```

These lines determine the coordinates of the four nearest pixels in the source image. The floor function rounds down to the nearest integer, giving the coordinates of the top-left pixel (x_0, y_0). The other three pixels are (x_1, y_0), (x_0, y_1) and (x_1, y_1).

The if checks if the nearest pixels are within the bounds of the source image. If they are out of bounds, the destination pixel is set to black (0).

calculate the distances from the transformed coordinates to the nearest pixel (x_0, y_0). dx is the horizontal distance, and dy is the vertical distance.

retrieve the intensity values of the four nearest pixels from the source image.

The first two lines interpolate the pixel intensities horizontally. The last line interpolates these values vertically to get the final intensity I

The round function is for converting the floating-point intensity value to the nearest int

31.4 – “NearestNeighborInterpolation” function – **part 1**

```
void AffineTransformNearestNeighbor(unsigned char src[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS], unsigned char dst[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS])
{
    double Sx = 1.13;
    double Sy = 2.13;
    int moveLeft = -40;
    int x = 20;
    int y = 10;
    double theta = -17.0 * M_PI / 180.0;

    double cos_theta = cos(theta);
    double sin_theta = sin(theta);

    // Affine transformation matrix components
    double a11 = Sx * cos_theta;
    double a12 = -Sx * sin_theta;
    double a21 = Sy * sin_theta;
    double a22 = Sy * cos_theta;

    int newRow, newCol;
    double tempRow, tempCol;

    for (int row = 0; row < NUMBER_OF_ROWS; row++)
    {
        for (int col = 0; col < NUMBER_OF_COLUMNS; col++)
        {
            // Apply the transformation: translate, rotate, scale, translate back
            tempCol = col + moveLeft; // Move 40 pixels left
            tempRow = row;

            // Rotation and scaling around point (20, 10)
            double transformedCol = a11 * (tempCol - x) + a12 * (tempRow - y) + x;
            double transformedRow = a21 * (tempCol - x) + a22 * (tempRow - y) + y;

            // Nearest neighbor interpolation: round to nearest integer
            newCol = round(transformedCol);
            newRow = round(transformedRow);

            // Copy pixel value if within bounds
            if (newCol >= 0 && newCol < NUMBER_OF_COLUMNS && newRow >= 0 && newRow < NUMBER_OF_ROWS)
            {
                dst[row][col] = src[newRow][newCol];
            }
            else
            {
                dst[row][col] = 0; // Assign black color to pixels out of bounds
            }
        }
    }
}
```

The end of the function:

```
48
49
50 // Copy pixel value if within bounds
51 if (newCol >= 0 && newCol < NUMBER_OF_COLUMNS && newRow >= 0 && newRow < NUMBER_OF_ROWS)
52 {
53     dst[row][col] = src[newRow][newCol];
54 }
55 else
56 {
57     dst[row][col] = 0; // Assign black color to pixels out of bounds
58 }
59
60 }
```

31.5 – “AffineTransformBilinear” function – part 1

```
62 void AffineTransformBilinear(unsigned char src[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS], unsigned char dst[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]
63 {
64     double Sx = 1.13;
65     double Sy = 2.13;
66     int moveLeft = -40;
67     int x = 20;
68     int y = 10;
69     double theta = -17.0 * M_PI / 180.0;
70
71     double cos_theta = cos(theta);
72     double sin_theta = sin(theta);
73
74     // Affine transformation matrix components
75     double a11 = Sx * cos_theta;
76     double a12 = -Sx * sin_theta;
77     double a21 = Sy * sin_theta;
78     double a22 = Sy * cos_theta;
79
80     double tempRow, tempCol;
81     int newRow, newCol;
82
83     for (int row = 0; row < NUMBER_OF_ROWS; row++)
84     {
85         for (int col = 0; col < NUMBER_OF_COLUMNS; col++)
86         {
87             // Apply the transformation: translate, rotate, scale, translate back
88             tempCol = col + moveLeft; // Move 40 pixels left
89             tempRow = row;
90
91             // Rotation and scaling around point (20, 10)
92             double transformedCol = a11 * (tempCol - x) + a12 * (tempRow - y) + x;
93             double transformedRow = a21 * (tempCol - x) + a22 * (tempRow - y) + y;
94
95             // Calculate the coordinates of the four nearest pixels
96             int x0 = floor(transformedCol);
97             int x1 = x0 + 1;
98             int y0 = floor(transformedRow);
```

31.6 – “AffineTransformBilinear” function – part 2

```
// Ensure the coordinates are within the bounds of the source image
if (x0 >= 0 && x1 < NUMBER_OF_COLUMNS && y0 >= 0 && y1 < NUMBER_OF_ROWS)
{
    // Calculate the distances
    double dx = transformedCol - x0;
    double dy = transformedRow - y0;

    // Get the values of the four nearest pixels
    double I00 = src[y0][x0];
    double I10 = src[y0][x1];
    double I01 = src[y1][x0];
    double I11 = src[y1][x1];

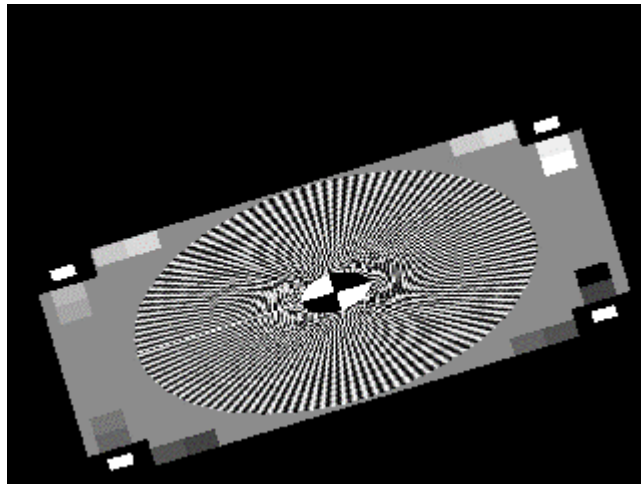
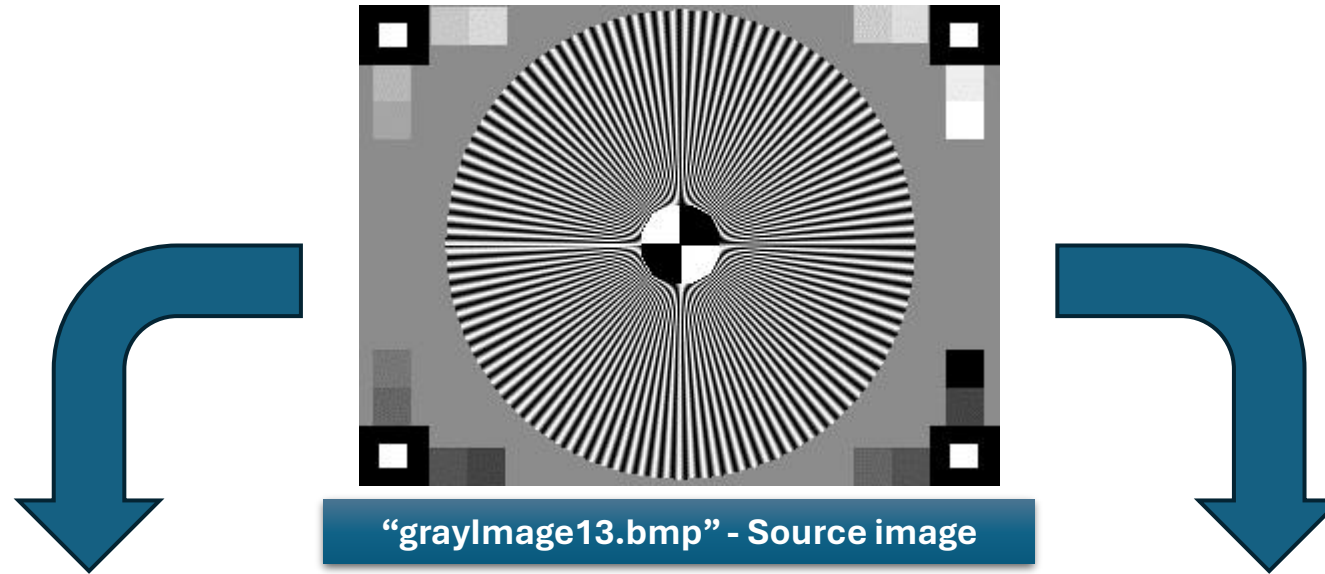
    // Perform bilinear interpolation
    double I0 = I00 * (1 - dx) + I10 * dx;
    double I1 = I01 * (1 - dx) + I11 * dx;
    double I = I0 * (1 - dy) + I1 * dy;

    // Assign the interpolated value to the destination image
    dst[row][col] = round(I);
}
else
{
    dst[row][col] = 0; // Assign black color to pixels out of bounds
}
```

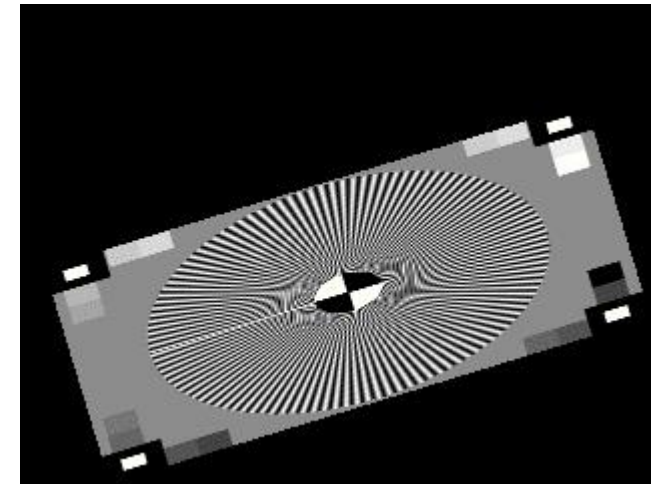
31.7 – the “main” function

```
124  void main()  
125  {  
126      unsigned char GrayImage[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];  
127      unsigned char NearestNeighborImage[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];  
128      unsigned char BilinearImage[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];  
129  
130      LoadGrayImageFromGrayBmpFile(GrayImage, "grayImage13.bmp");  
131      AffineTransformNearestNeighbor(GrayImage, NearestNeighborImage);  
132      StoreGrayImageAsGrayBmpFile(NearestNeighborImage, "A31.bmp");  
133  
134      LoadGrayImageFromGrayBmpFile(GrayImage, "grayImage13.bmp");  
135      AffineTransformBilinear(GrayImage, BilinearImage);  
136      StoreGrayImageAsGrayBmpFile(BilinearImage, "A32.bmp");  
137  
138      WaitForUserPressKey();  
139  }  
140
```

31.8 - Resulted photographs – “A31.bmp”, “A32.bmp”



“A31.bmp” - Created by using nearest neighbor interpolation

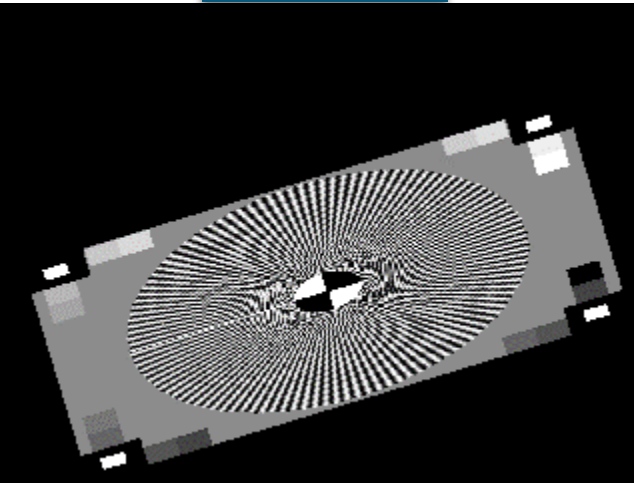


“A32.bmp” - Created by using bilinear interpolation

31.9 –profilers of Resulted photographs – “A31.bmp”, “A32.bmp”

13

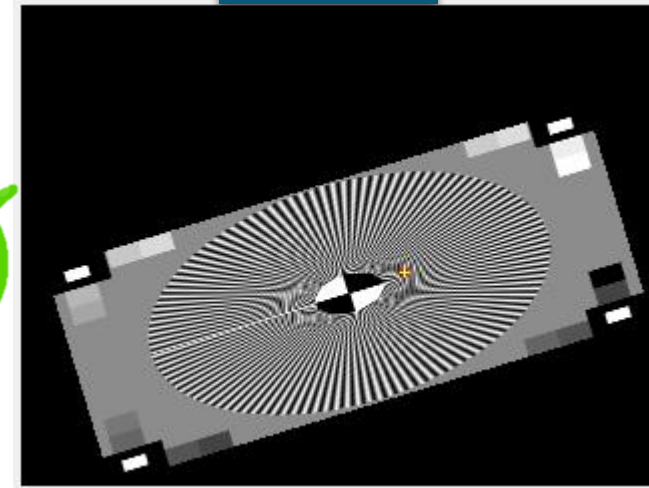
A31.bmp



Point:	Mouse	Expl.	(M-E)
Row	106	106	0
Column	191	191	0
Red	248	248	0
Green	248	248	0
Blue	248	248	0

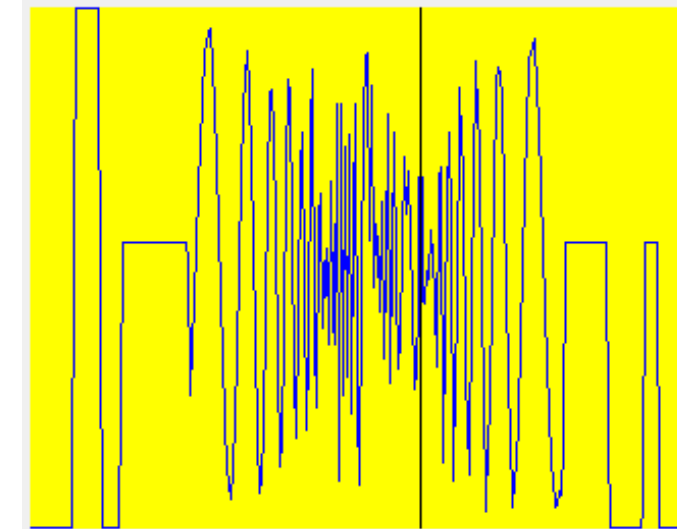
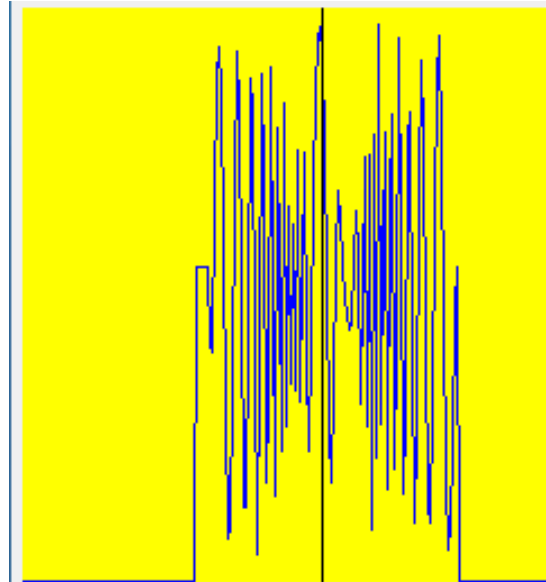
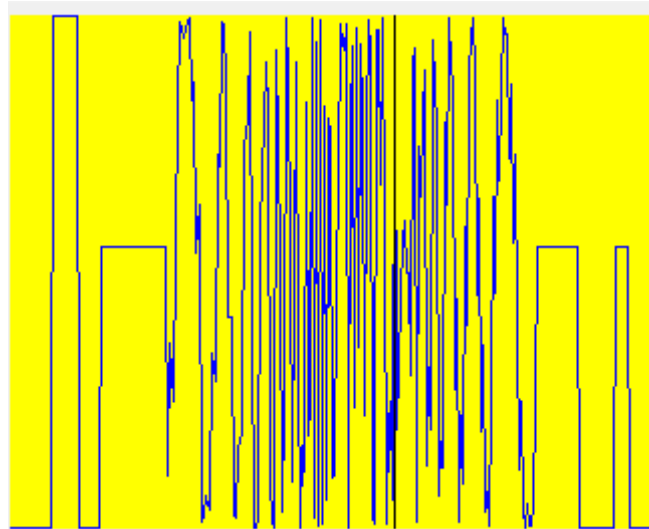
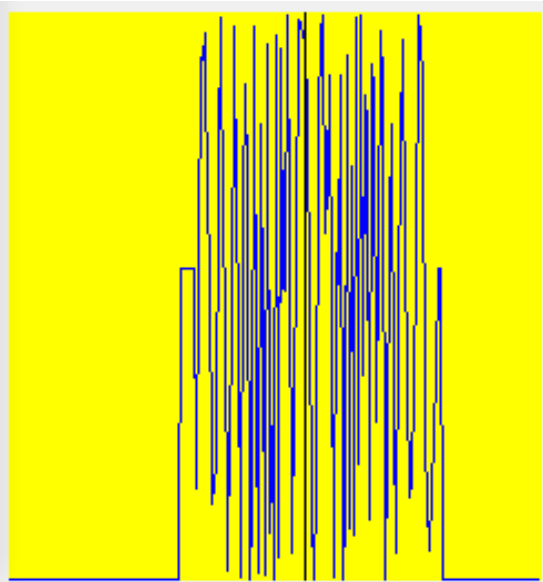


A32.bmp



Point:	Mouse	Expl.	(M-E)
Row	106	106	0
Column	191	191	0
Red	234	234	0
Green	234	234	0
Blue	234	234	0

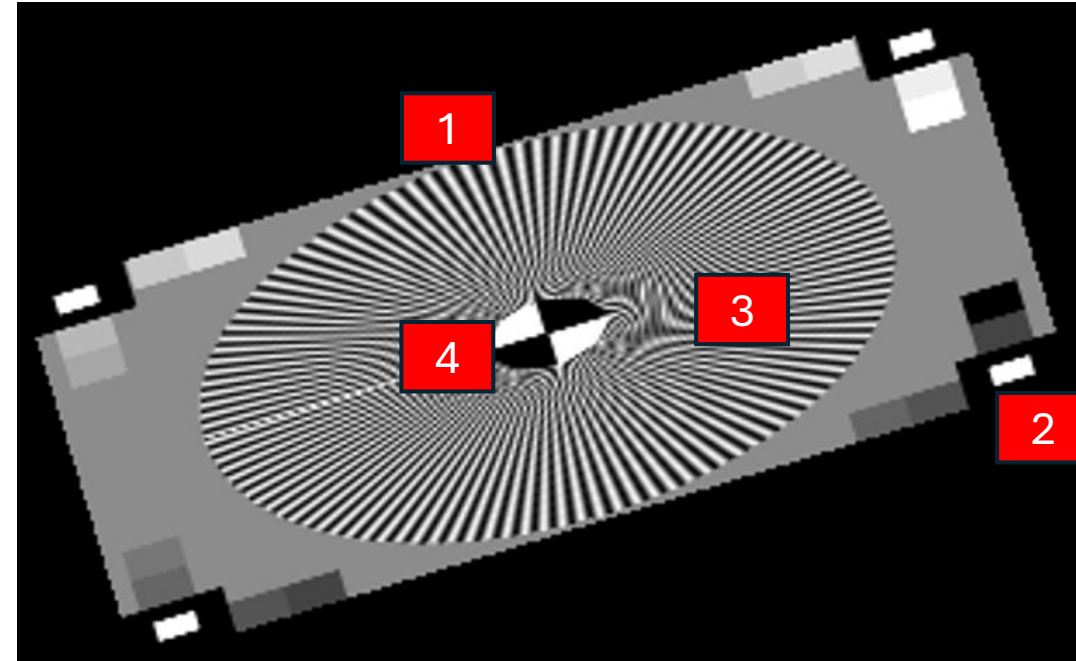
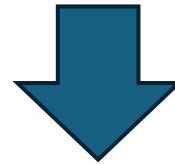
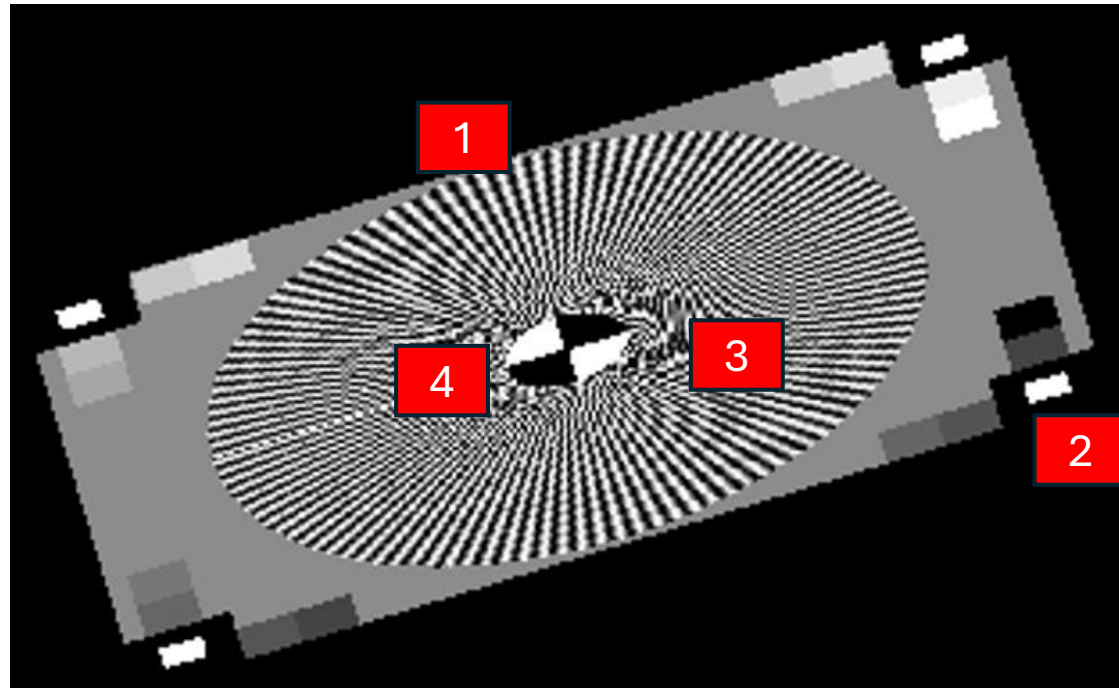
Conclusion: “A32.bmp” looks smoother than “A31.bmp”, since we applied on it the bilinear interpolation, it can be seen in the profilers graphs. We can see that both column (left) and the row (right) have more noise in A31.bmp



31.10 –deeper look at the results-“A31.bmp” vs “A32.bmp”

A31.bmp

A32.bmp



1. The black straight lines are much more disrupted in the A31.bmp.
2. The small white rectangles in the corners – their outline are much more disrupted in the A31.bmp.
3. The area from the right hand side of the center is much more disrupted – a lot of added noise in A31.bmp.
4. The lines that assemble the black and white circle in the center are much more disrupted in A31.bmp.

Therefore, our opinion is that the bilinear interpolation is a way more successful method than the nearest neighbor interpolation.

31.8 – What did we learn ?

- 1) To use two algorithms that can create the same result in different ways: Bilinear interpolation and nearest neighbor.
- 2) To acknowledge what is interpolation and how it use it.
- 3) How to use Nearest Neighbor Interpolation algorithm to rotate the image.
- 4) How to use Bilinear Interpolation algorithm to rotate the image.
- 5) To apply sequence of operations on an image (move it, scale it, rotate it around the point).